

LAB 2: CLASS, PROPERTIES AND METHOD IN JAVA

1. Java method overloading

Method Overloading in Java is a technique that allows a class to have **multiple methods with the same name but different parameters**.

Overloaded methods must have the same name but differ in:

- Number of parameters
- Data type of parameters
- Order of parameter data types

Note: Method overloading **does not depend on the return type**. Changing only the return type while keeping parameters the same will cause an error.

Benefits of Method Overloading:

- Improves code readability by using the same method name for similar tasks.
- Increases flexibility by allowing a method to be called in different ways.
- Optimizes performance by reducing the need to create multiple method names for similar operations.

Example: Advantages of Method Overloading before and after implementation

Let's consider a simple **calculator** program that performs addition and multiplication.

Before applying method overloading

Without method overloading, we must define separate method names for each operation, making the code less readable and harder to maintain.

```

class Calculator {
    // Addition methods with different names
    1 usage
    int addInt(int a, int b) {
        return a + b;
    }

    1 usage
    double addDouble(double a, double b) {
        return a + b;
    }

    // Multiplication methods with different names
    1 usage
    int multiplyInt(int a, int b) {
        return a * b;
    }

    1 usage
    double multiplyDouble(double a, double b) {
        return a * b;
    }
}

public class MethodOverloading {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        System.out.println(calc.addInt(a: 5, b: 10));           // Output: 15
        System.out.println(calc.addDouble(a: 5.5, b: 10.2));   // Output: 15.7
        System.out.println(calc.multiplyInt(a: 4, b: 3));       // Output: 12
        System.out.println(calc.multiplyDouble(a: 4.5, b: 2.5)); // Output: 11.25
    }
}

```

Problems:

- Each method needs a different name (*addInt*, *addDouble*, *multiplyInt*, *multiplyDouble*).
- Code is **less readable and harder to maintain**.
- If we need to support more data types, we must create **more methods** with different names.

After applying method overloading

With method overloading, we can use **the same method name** for similar operations, making the code cleaner and more maintainable.

```
class Calculator {
    // Overloaded addition methods
    1 usage
    int add(int a, int b) {
        return a + b;
    }

    1 usage
    double add(double a, double b) {
        return a + b;
    }

    // Overloaded multiplication methods
    1 usage
    int multiply(int a, int b) {
        return a * b;
    }

    1 usage
    double multiply(double a, double b) {
        return a * b;
    }
}

public class MethodOverloading {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        System.out.println(calc.add(a: 5, b: 10)); // Output: 15
        System.out.println(calc.add(a: 5.5, b: 10.2)); // Output: 15.7
        System.out.println(calc.multiply(a: 4, b: 3)); // Output: 12
        System.out.println(calc.multiply(a: 4.5, b: 2.5)); // Output: 11.25
    }
}
```

After applying method overloading:

- Improved readability – The method names are the same, making the code more intuitive.
- Easier maintenance – If we need to extend functionality, we simply add another overloaded method.
- Less redundant code – We avoid defining multiple method names for similar tasks.

2. Class and object

Class

- A class is a **blueprint or template** for creating objects.
- It **defines attributes (properties) and behaviors (methods)** that describe an object.
- A class does not occupy memory until an object is created.

Example:

A class "Car" can have **attributes** such as color, brand, speed, and **methods** like drive, stop, accelerate.

```
class Car {  
    String brand;  
    int speed;  
  
    Car(String brand, int speed) {  
        this.brand = brand;  
        this.speed = speed;  
    }  
  
    void display() {  
        System.out.println("Brand: " + brand + ", Speed: " + speed + " km/h")  
    }  
}
```

Object

- An object is a **specific instance** created from a class.
- Each object has its own values for the attributes defined in the class.
- Objects are created in memory using the new keyword.

Example:

Creating objects from the Car class

```
public class Main {  
    public static void main(String[] args) {  
        Car car1 = new Car( brand: "Toyota", speed: 120);  
        Car car2 = new Car( brand: "BMW", speed: 150);  
  
        car1.display();  
        car2.display();  
    }  
}
```

Explanation:

- **car1** is an object of the Car class with *brand* = "Toyota" and *speed* = 120.
- **car2** is another object with *brand* = "BMW" and *speed* = 150.

Relationship Between Class and Object

The relationship between **Class** and **Object** in Object-Oriented Programming can be summarized as follows:

Class is a **blueprint**, object is **an instance created from that blueprint**.

A class can create **multiple objects**, but **each object belongs to only one class**.

A class **defines structure and behavior**, and objects **store specific data and perform actions**.

A class does not occupy memory, but objects do when created.

Example:

A Dog class can be used to create multiple dog objects.

```
class Dog {
    String name;
    int age;

    Dog(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void bark() {
        System.out.println(name + " is barking!");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Dog dog1 = new Dog( name: "Buddy", age: 3);
        Dog dog2 = new Dog( name: "Max", age: 5);

        dog1.bark();
        dog2.bark();
    }
}
```

3. Class in Java

In Java, a **class** is a **blueprint or template** for creating objects. A class can contain:

- **Properties (attributes)** – which describe the characteristics of an object.
- **Methods** – which describe the behavior of an object.

A class does not occupy memory until an object is created from it.

3.1 Class properties

Class properties are declared within a class and represent the **state of an object**.

Example:

```
class Student {  
    String name;  
    int age;  
}
```

Each object of the Student class will have its own values for name and age.

Initializing objects and using properties:

```
public class Main {  
    public static void main(String[] args) {  
        Student student1 = new Student();  
        student1.name = "Minh";  
        student1.age = 20;  
  
        Student student2 = new Student();  
        student2.name = "Lan";  
        student2.age = 22;  
  
        System.out.println(student1.name + " - " + student1.age);  
        System.out.println(student2.name + " - " + student2.age);  
    }  
}
```

3.2 The “this” keyword in Java

The this keyword is used to refer to the current instance of the class.

Uses of this

- Differentiates between instance variables and local variables when they have the same name.
- Calls another constructor within the same class (this()).
- Returns the current instance of the class.

Example:

```
class Person {
    String name;

    Person(String name) {
        this.name = name; // 'this.name' refers to the instance variable, while 'name' is the local variable
    }

    void display() {
        System.out.println("Name: " + this.name);
    }
}

public class Main {
    public static void main(String[] args) {
        Person p1 = new Person( name: "Huy");
        p1.display();
    }
}
```

Explanation:

- this.name = name; helps distinguish between the instance variable (this.name) and the local variable (name).
- Without this, the program would not be able to differentiate between them.

3.3 Methods in Java classes

Methods define the **actions** that an object can perform.

Declaring a Method

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
}
```

Explanation:

- `int add(int a, int b)`: This method takes two parameters a and b and returns their sum.

3.4 Calling a method in Java

```
public class Main {  
    public static void main(String[] args) {  
        Calculator cal = new Calculator();  
        int result = cal.add(a: 5, b: 10);  
        System.out.println("Sum: " + result);  
    }  
}
```

3.5 Constructor in Java

A **constructor** in Java is a special method that is called when an object of a class is created. Its purpose is to **initialize the object** with default or specific values.

Characteristics of a Constructor

- It has the **same name** as the class.
- It **does not have a return type** (not even void).
- It is **automatically called** when an object is created.
- If no constructor is defined, Java provides a **default constructor**.

Types of constructors:

- Default constructor

A **default constructor** has no parameters and initializes instance variables with default values.

Example:

```
class Car {
    String brand;
    int speed;

    // Default constructor
    Car() {
        brand = "Unknown";
        speed = 0;
    }

    void display() {
        System.out.println("Brand: " + brand + ", Speed: " + speed + " km/h");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Car car1 = new Car(); // Calls the default constructor
        car1.display();
    }
}
```

- Parameterized constructor

A **parameterized constructor** allows passing values during object creation.

Example:

```
class Car {
    String brand;
    int speed;

    // Parameterized constructor
    Car(String b, int s) {
        brand = b;
        speed = s;
    }

    void display() {
        System.out.println("Brand: " + brand + ", Speed: " + speed + " km/h");
    }
}
```

```

public class Main {
    public static void main(String[] args) {
        Car car1 = new Car( b: "Toyota", s: 120);
        Car car2 = new Car( b: "Honda", s: 150);

        car1.display();
        car2.display();
    }
}

```

- Constructor overloading

Constructor overloading means defining multiple constructors with different parameters to provide flexibility.

Example:

```

class Car {
    String brand;
    int speed;

    // Default constructor
    Car() {
        brand = "Unknown";
        speed = 0;
    }

    // Constructor with one parameter
    Car(String b) {
        brand = b;
        speed = 0;
    }

    // Constructor with two parameters
    Car(String b, int s) {
        brand = b;
        speed = s;
    }

    void display() {
        System.out.println("Brand: " + brand + ", Speed: " + speed + " km/h");
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        Car car1 = new Car();
        Car car2 = new Car( b: "Ford");
        Car car3 = new Car( b: "BMW", s: 180);

        car1.display();
        car2.display();
        car3.display();
    }
}

```

Exercise:

1. Fraction Class

Create a **Fraction** class with the following attributes:

- **Numerator** and **Denominator**

Methods include:

- Default constructor, parameterized constructor (Ensure fraction validity before initialization).
- Method to **simplify** the fraction.
- Method to **add** two fractions.
- Method to **subtract** two fractions.
- Method to **multiply** two fractions.
- Method to **divide** two fractions.

Implement the main method to **execute all above methods**.

2. Time Class

Create a **Time** class with attributes:

- **Hour, Minute, and Second**

Methods include:

- Default constructor, parameterized constructor (Ensure valid time values before initialization).
- Method to **add** hours to the current time and return the updated result.
- Method to **subtract** hours from the current time and return the updated result.

Implement the main method to **execute all above methods**.

3. Circle Class

Create a **Circle** class to define a circle **C(O, r)** with:

- **Center O(a, b)** and **Radius r**

Methods include:

- **area()** → Calculate the area of the circle.
- **perimeter()** → Calculate the perimeter of the circle.
- **testBelongs(x, y)** → Check if point A(x, y) lies inside the circle **C(O, r)** or not.

Implement the main method to **execute all above methods**.

4. Dice Class

Create a **Dice** class to model a six-sided dice and its rolling behavior.

- Each **Dice** object has an attribute **num**, which stores the most recent roll value (a number between **1 and 6**).
- Implement a **roll()** method to simulate rolling the dice and return the outcome.
- Simulate rolling the dice **n** times (where **n** is input by the user).
- Calculate the frequency of each face (1-6) and compute the probability of occurrence for each face.

Implement the main method to **execute all above methods**.

5. First-Degree Polynomial Class

Create a **FirstDegreePolynomial** class to represent polynomials of the form:

F(x) = ax + b (where **a ≠ 0**).

Methods include:

- A method to evaluate **F(x)** for a given **x₀**.
- A method to find the **root** of the polynomial (solve **F(x) = 0**).
- A method to **add** two first-degree polynomials and return the result.

Implement the main method to **execute all above methods**.

6. Author and Book Management

A program is required to **manage book publishing by authors**.

Author Class

Attributes:

- **Name, Age, Address**
- **A list of published books (List<Book>)**

Methods:

- Default constructor, parameterized constructor.

Book Class

Attributes:

- **Title, Year of Publication, Genre, Price, Number of Copies Sold**

Methods:

- Default constructor, parameterized constructor, and no-argument constructor.

Additional Requirements

Implement methods in the **Author** class to:

- **Calculate the total revenue** from sold books.
- **Calculate revenue by genre** (sum of sales per genre).
- **Calculate revenue by year** (sum of sales per year).

Implement the main method to **execute all above methods**.

7. UITStudent Class

Create a **UITStudent** class with attributes:

- Basic **student card information** and **citizen identification** details (duplicate information can be omitted).
- **GPA (Grade Point Average)**

Methods include:

- Default constructor, parameterized constructor.
- A method to **input student details**.
- A method to **display student personal information**.
- A method to **calculate the student's current age** based on the system date.
- A method to **calculate the expected graduation date**, based on a parameter "CTDT", which accepts values:
 - "CQ" → 4 years
 - "BCU" → 3.5 years
 - "CTTT" → 4.5 years
 - The assumed **start year** = **Birth Year** + **18**, with an enrollment month of **September**.

Create a list of 5 UITStudent objects, input their details, and perform the following:

- Print the **age** and **graduation date** of all students in the list.
- Identify the **student with the highest GPA**.
- Print the **graduation order** of students based on their expected graduation year.

Implement the main method to **execute all above methods**.